

Hướng dẫn Topological Data Analysis (TDA) cho NLP

Mục đích và ứng dụng

Topological Data Analysis (TDA) giúp phân tích cấu trúc hình học của dữ liệu văn bản thông qua:

- **Persistent Homology:** Tìm hiểu các "lỗ" và "thành phần liên thông" trong không gian embedding
- **Betti Numbers:** Đo lường độ phức tạp topological của dữ liệu
- **Ứng dụng:** Phân loại sentiment, phát hiện chủ đề, phân tích độ tương tự văn bản

Giải thích từng bước code

1. Import thư viện cần thiết

```
python

import numpy as np
from sentence_transformers import SentenceTransformer
from sklearn.preprocessing import StandardScaler
from ripser import ripser
from persim import plot_diagrams
```

Giải thích:

- `sentence_transformers`: Chuyển đổi văn bản thành vector embedding
- `ripser`: Thư viện tính persistent homology
- `persim`: Vẽ persistence diagrams
- `StandardScaler`: Chuẩn hóa dữ liệu

2. Chuẩn bị dữ liệu và tạo embeddings

```
python

texts = ["Sản phẩm tuyệt vời!", "Dịch vụ tệ!", "Món ăn ngon, phục vụ chậm."]
model = SentenceTransformer('distiluse-base-multilingual-cased-v2')
embeddings = model.encode(texts)
```

Giải thích:

- Sử dụng mô hình đa ngôn ngữ để tạo vector embedding 512 chiều
- Mỗi câu được biểu diễn bằng một vector số thực

3. Chuẩn hóa dữ liệu

```
python
```

```
scaler = StandardScaler()
embeddings_scaled = scaler.fit_transform(embeddings)
```

Tại sao cần chuẩn hóa?

- Đảm bảo các chiều có cùng scale
- Cải thiện độ chính xác của thuật toán TDA
- Loại bỏ bias do các feature có độ lớn khác nhau

4. Tính Persistent Homology

```
python
```

```
result = ripser(embeddings_scaled, maxdim=2)
diagrams = result['dgms']
```

Persistent Homology là gì?

- Phân tích cấu trúc topological qua nhiều mức độ resolution
- `maxdim=2`: Tính đến dimension 2 (connected components, loops, voids)
- Kết quả là persistence diagrams cho từng dimension

5. Tính Betti Numbers

```
python
```

```
betty_0 = sum(1 for p in diagrams[0] if p[1] != np.inf)
betty_1 = sum(1 for p in diagrams[1] if p[1] != np.inf)
```

Betti Numbers có ý nghĩa gì?

- β_0 : Số thành phần liên thông (connected components)
- β_1 : Số loops/holes trong dữ liệu
- β_2 : Số voids (cavities) 3D

6. Tạo features kết hợp

```
python
```

```
betty_features = np.array([betty_0, betty_1] * len(texts))
combined_features = np.concatenate([embeddings, betty_features], axis=1)
```

Mục đích: Kết hợp thông tin semantic (embeddings) với thông tin topological (Betti numbers)

Cách sử dụng thực tế

Ví dụ 1: Phân tích sentiment

python

```
def analyze_sentiment_with_tda(texts):
    # Tạo embeddings
    model = SentenceTransformer('distiluse-base-multilingual-cased-v2')
    embeddings = model.encode(texts)

    # Chuẩn hóa
    scaler = StandardScaler()
    embeddings_scaled = scaler.fit_transform(embeddings)

    # Tính TDA features
    result = ripser(embeddings_scaled, maxdim=1)
    diagrams = result['dgms']

    # Tính persistence features
    persistence_0 = [p[1] - p[0] for p in diagrams[0] if p[1] != np.inf]
    persistence_1 = [p[1] - p[0] for p in diagrams[1] if p[1] != np.inf]

    return {
        'avg_persistence_0': np.mean(persistence_0) if persistence_0 else 0,
        'max_persistence_1': np.max(persistence_1) if persistence_1 else 0,
        'num_components': len(diagrams[0]),
        'num_loops': len(diagrams[1])
    }

# Sử dụng
texts = [
    "Sản phẩm rất tốt, tôi hài lòng!",
    "Dịch vụ tệ, không khuyên dùng",
    "Bình thường, không có gì đặc biệt"
]
tda_features = analyze_sentiment_with_tda(texts)
print(tda_features)
```

Ví dụ 2: So sánh độ phức tạp của documents

python

```
def compare_document_complexity(doc1_sentences, doc2_sentences):
    def get_complexity_score(sentences):
        model = SentenceTransformer('distiluse-base-multilingual-cased-v2')
        embeddings = model.encode(sentences)

        if len(embeddings) < 3: # Cần ít nhất 3 điểm
            return 0

        scaler = StandardScaler()
        embeddings_scaled = scaler.fit_transform(embeddings)

        result = ripser(embeddings_scaled, maxdim=1)
        diagrams = result['dgms']

        # Tính complexity score dựa trên persistence
        complexity = 0
        for dim in range(len(diagrams)):
            for birth, death in diagrams[dim]:
                if death != np.inf:
                    complexity += (death - birth)

        return complexity

    score1 = get_complexity_score(doc1_sentences)
    score2 = get_complexity_score(doc2_sentences)

    return {
        'doc1_complexity': score1,
        'doc2_complexity': score2,
        'more_complex': 'Document 1' if score1 > score2 else 'Document 2'
    }
```

Cài đặt thư viện

bash

```
pip install sentence-transformers
pip install scikit-learn
pip install ripser
pip install persim
pip install matplotlib
```

Lưu ý quan trọng

Giới hạn và thách thức:

1. **Kích thước dữ liệu:** TDA tốn nhiều bộ nhớ với datasets lớn
2. **Thời gian tính toán:** Có thể chậm với high-dimensional data
3. **Tulkuabilitas:** Cần kinh nghiệm để diễn giải kết quả

Tối ưu hóa:

1. **Dimension reduction:** Sử dụng PCA trước khi áp dụng TDA
2. **Sampling:** Lấy mẫu con cho datasets lớn
3. **Parallel processing:** Sử dụng GPU nếu có thể

Ứng dụng nâng cao

1. Topic Modeling với TDA

- Sử dụng persistent homology để phát hiện cụm chủ đề
- Betti numbers giúp xác định số topics tối ưu

2. Document Similarity

- So sánh persistence diagrams giữa các documents
- Tính Wasserstein distance giữa các diagrams

3. Anomaly Detection

- Phát hiện văn bản có cấu trúc topological bất thường
- Sử dụng trong spam detection, fake news detection

Kết luận

TDA cung cấp góc nhìn mới về cấu trúc dữ liệu văn bản, bổ sung cho các phương pháp NLP truyền thống. Khi kết hợp với embeddings, nó có thể cải thiện performance của nhiều tác vụ NLP, đặc biệt trong phân tích sentiment và document classification.